

03_backend-requirements

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ backend/  
    └─ requirements.txt
```

El archivo `requirements.txt` se encuentra dentro de la carpeta `backend/`.

Su función es declarar las dependencias Python necesarias para ejecutar el backend del laboratorio SIEM MVP. Estas dependencias se instalan dentro del contenedor de la API durante la construcción de la imagen Docker.

Este archivo está directamente relacionado con el `Dockerfile`, concretamente con esta línea:

```
RUN pip install --no-cache-dir -r requirements.txt
```

Esto significa que Docker usa `requirements.txt` como lista de paquetes que deben instalarse para que el backend pueda funcionar.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,220p' backend/requirements.txt
```

Este comando muestra el contenido del archivo `backend/requirements.txt`.

Desglose:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para leer o transformar texto.

```
-n
```

Evita que `sed` imprima todo el archivo automáticamente.

```
'1,220p'
```

Indica que se impriman las líneas de la 1 a la 220.

```
backend/requirements.txt
```

Ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
fastapi  
uvicorn  
sqlalchemy>=2.0  
psycopg[binary]  
alembic  
pytest  
httpx
```

4 Función general del archivo

El archivo `requirements.txt` define las librerías externas que necesita el backend para funcionar.

En Python, las aplicaciones suelen dividirse entre:

```
Código propio del proyecto
+
Dependencias externas instaladas con pip
```

En este laboratorio, el código propio está en:

```
backend/app/
```

y las dependencias externas están declaradas en:

```
backend/requirements.txt
```

Durante la construcción de la imagen Docker, el archivo se copia dentro del contenedor:

```
COPY requirements.txt /app/requirements.txt
```

Después se instalan las dependencias:

```
RUN pip install --no-cache-dir -r requirements.txt
```

Por tanto, este archivo es esencial para que el contenedor `siem-api` tenga disponibles FastAPI, Uvicorn, SQLAlchemy, Alembic, Pytest y el resto de herramientas necesarias.

5 Estructura general del archivo

El archivo contiene una dependencia por línea:

```
fastapi
uvicorn
sqlalchemy>=2.0
psycopg[binary]
alembic
pytest
httpx
```

Cada línea representa un paquete que `pip` debe instalar.

Algunas dependencias aparecen sin versión concreta:

```
fastapi
uvicorn
alembic
pytest
httpx
```

Esto significa que `pip` instalará una versión compatible disponible en el momento de la instalación.

Otras dependencias incluyen restricciones o extras:

```
sqlalchemy>=2.0
psycopg[binary]
```

Estas líneas tienen una sintaxis más específica y conviene entenderlas bien.

6 Análisis línea por línea

Línea 1: `fastapi`

```
fastapi
```

`fastapi` es el framework principal utilizado para construir la API del backend.

Un framework web permite definir rutas HTTP, recibir peticiones, validar datos y devolver respuestas.

En este proyecto, FastAPI se usa para crear endpoints como:

```
/health
/info
/ingest
/events
/rules
/alerts
/metrics
```

FastAPI permite construir una API REST de forma clara mediante funciones Python y decoradores.

Ejemplo conceptual:

```
@app.get("/health")
def health_check():
    return {"status": "ok"}
```

En este ejemplo:

```
@app.get("/health")
```

indica que la función responderá a peticiones HTTP `GET` en la ruta `/health`.

La dependencia `fastapi` es imprescindible porque el archivo principal del backend, normalmente `backend/app/main.py`, crea una aplicación con una estructura similar a:

```
app = FastAPI()
```

Sin esta dependencia, el backend no podría importar FastAPI y aparecería un error similar a:

```
ModuleNotFoundError: No module named 'fastapi'
```

Línea 2: `uvicorn`

```
uvicorn
```

`uvicorn` es el servidor ASGI encargado de ejecutar la aplicación FastAPI.

FastAPI define la lógica de la aplicación, pero necesita un servidor que escuche peticiones HTTP reales. Ese papel lo cumple Uvicorn.

En el `Dockerfile` y en `docker/compose.yml`, Uvicorn aparece en el comando de arranque:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Desglose funcional:

<code>uvicorn</code>	→ servidor que arranca la aplicación
<code>app.main:app</code>	→ ubicación de la instancia FastAPI
<code>--reload</code>	→ recarga automática en desarrollo

```
--host 0.0.0.0 → escucha dentro del contenedor en todas las interfaces
--port 8000 → puerto interno de la API
```

Sin `uvicorn`, FastAPI estaría instalado, pero no habría servidor para ejecutar la aplicación.

El error típico sería:

```
uvicorn: command not found
```

O:

```
ModuleNotFoundError: No module named 'uvicorn'
```

Línea 3: `sqlalchemy>=2.0`

```
sqlalchemy>=2.0
```

`sqlalchemy` es la librería utilizada para trabajar con la base de datos desde Python.

En este proyecto, SQLAlchemy cumple el papel de ORM.

ORM significa:

```
Object Relational Mapper
```

Es decir, una herramienta que permite representar tablas de base de datos como clases Python.

Por ejemplo, en vez de escribir siempre SQL manual como:

```
SELECT * FROM events;
```

el proyecto puede trabajar con modelos Python como:

```
Event
Rule
Alert
```

La parte:

```
>=2.0
```

es una restricción de versión.

Desglose:

```
sqlalchemy → nombre del paquete
>=          → mayor o igual que
2.0         → versión mínima requerida
```

Esto significa:

```
Instala SQLAlchemy en una versión igual o superior a la 2.0.
```

Se fuerza como mínimo la versión 2.0 porque SQLAlchemy 2 introdujo una forma moderna de trabajar con sesiones, modelos y consultas.

En este proyecto, SQLAlchemy se relaciona con archivos como:

```
backend/app/db/database.py
backend/app/db/session.py
backend/app/db/base.py
backend/app/models/event.py
```

```
backend/app/models/rule.py
backend/app/models/alert.py
```

Sin SQLAlchemy, el backend no podría definir los modelos ni comunicarse correctamente con PostgreSQL mediante ORM.

Línea 4: `psycopg[binary]`

```
psycopg[binary]
```

`psycopg` es el driver que permite que Python se conecte a PostgreSQL.

FastAPI y SQLAlchemy no hablan directamente con PostgreSQL por sí solos. Necesitan un driver de conexión.

La relación sería:

```
FastAPI
  ↓
SQLAlchemy
  ↓
psycopg
  ↓
PostgreSQL
```

La parte:

```
[binary]
```

es un extra del paquete.

En Python, algunos paquetes permiten instalar funcionalidades adicionales usando corchetes.

La estructura es:

```
paquete[extra]
```

En este caso:

```
psycopg[binary]
```

significa:

```
Instala psycopg junto con su variante binaria preparada.
```

La variante binaria facilita la instalación porque evita tener que compilar componentes nativos o depender de librerías del sistema durante la instalación.

Esto es especialmente útil dentro de Docker, porque una imagen ligera como:

```
python:3.12-slim
```

no siempre trae herramientas de compilación completas.

Sin `psycopg`, SQLAlchemy no podría conectarse a PostgreSQL usando una URL de conexión como:

```
postgresql+psycopg://usuario:password@db:5432/base_de_datos
```

Un error típico sería:

```
ModuleNotFoundError: No module named 'psycopg'
```

o un error de SQLAlchemy indicando que no encuentra el driver de PostgreSQL.

Línea 5: alembic

```
alembic
```

`alembic` es la herramienta utilizada para gestionar migraciones de base de datos.

Una migración es un archivo que describe cambios en la estructura de la base de datos.

Por ejemplo:

```
Crear tabla events
Añadir columna meta a events
Crear tabla rules
Crear tabla alerts
Añadir campo threshold a rules
Añadir campo status a alerts
```

En el proyecto, las migraciones están en:

```
backend/alembic/versions/
```

Ejemplos de archivos reales del proyecto:

```
b8f4b712e6b5_create_events_table.py
be0f61d66ed2_add_meta_to_events.py
d841bcb4d197_add_rules_and_alerts.py
b1b85630457f_add_threshold_to_rules.py
2e15d22277a_add_status_and_updated_at_to_alerts.py
```

Alembic permite evolucionar la base de datos sin tener que borrar y recrear todo manualmente.

La relación sería:

```
Modelos SQLAlchemy
    ↓
Alembic genera o aplica migraciones
    ↓
PostgreSQL actualiza su estructura
```

Sin Alembic, podrías usar la base de datos, pero sería más difícil controlar los cambios de esquema durante el desarrollo.

Línea 6: pytest

```
pytest
```

`pytest` es el framework utilizado para ejecutar tests automatizados en Python.

En este proyecto se relaciona con la carpeta:

```
backend/tests/
```

Concretamente, aparecen archivos como:

```
backend/tests/test_health.py
backend/tests/test_alerts_ui.py
```

Pytest permite comprobar automáticamente que ciertas partes del backend funcionan correctamente.

Por ejemplo, un test puede verificar que el endpoint `/health` responde correctamente.

Ejemplo conceptual:

```
def test_health():
    response = client.get("/health")
    assert response.status_code == 200
```

Desglose conceptual:

```
client.get("/health")
```

Simula una petición HTTP al endpoint `/health`.

```
assert response.status_code == 200
```

Comprueba que la respuesta tenga código HTTP 200.

Sin `pytest`, no se podrían ejecutar los tests con el comando:

```
pytest
```

El error típico sería:

```
pytest: command not found
```

Línea 7: `httpx`

```
httpx
```

`httpx` es una librería cliente HTTP para Python.

En proyectos con FastAPI, suele utilizarse en los tests para simular peticiones HTTP a la API.

Aunque el usuario no abra el navegador ni use `curl`, los tests pueden hacer peticiones internas a endpoints como:

```
/health
/alerts
/events
```

La relación habitual es:

```
pytest ejecuta el test
    ↓
httpx/TestClient simula una petición HTTP
    ↓
FastAPI procesa la petición
    ↓
el test comprueba la respuesta
```

En muchos proyectos FastAPI, `httpx` es necesario porque el cliente de pruebas se apoya en esta librería.

Sin `httpx`, algunos tests podrían fallar aunque la API esté correctamente programada.

Un error típico sería:

```
ModuleNotFoundError: No module named 'httpx'
```

7 Relación entre las dependencias

Estas dependencias no son independientes entre sí; forman una pila técnica.

La pila principal del backend sería:

```
uvicorn
  ↓ ejecuta
fastapi
  ↓ define endpoints y lógica API
sqlalchemy
  ↓ gestiona modelos y sesiones
psycopg[binary]
  ↓ conecta con
postgresql
```

La pila de base de datos y migraciones sería:

```
sqlalchemy
  ↓ define modelos
alembic
  ↓ aplica migraciones
postgresql
```

La pila de testing sería:

```
pytest
  ↓ ejecuta pruebas
httpx
  ↓ simula peticiones HTTP
fastapi
```

Por tanto, el archivo `requirements.txt` resume las piezas mínimas necesarias para que el backend pueda:

1. Arrancar la API.
2. Exponer endpoints HTTP.
3. Conectarse a PostgreSQL.
4. Gestionar modelos de datos.
5. Aplicar migraciones.
6. Ejecutar pruebas automatizadas.

8 Relación con Dockerfile

El archivo `requirements.txt` se utiliza directamente durante la construcción de la imagen Docker.

En el Dockerfile aparece:

```
COPY requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
```

Primero se copia el archivo dentro de la imagen:

```
backend/requirements.txt → /app/requirements.txt
```

Después, `pip` instala cada dependencia.

Este orden permite aprovechar la caché de Docker. Si solo cambia el código de la aplicación, pero no cambia `requirements.txt`, Docker no necesita reinstalar todas las dependencias en cada construcción.

Esto acelera mucho el desarrollo.

9 Relación con el flujo técnico del laboratorio

El archivo `requirements.txt` participa en el flujo de preparación del backend:

```
docker/compose.yml
↓
```

```
servicio api
↓
backend/Dockerfile
↓
COPY requirements.txt
↓
pip install -r requirements.txt
↓
instalación de FastAPI, Uvicorn, SQLAlchemy, Psycopg, Alembic, Pytest y HTTPX
↓
contenedor preparado para ejecutar app.main:app
```

Este archivo no procesa eventos ni genera alertas, pero sin él el backend no tendría las librerías necesarias para hacerlo.

10 Errores típicos relacionados

♦ Paquete mal escrito

Si una línea del archivo tiene un nombre incorrecto, `pip install` fallará.

Ejemplo:

```
fastappi
```

provocaría un error porque ese paquete no corresponde al framework esperado.

♦ Falta de versión fijada

En este archivo, la mayoría de dependencias no tienen versión fija:

```
fastapi
uvicorn
alembic
pytest
httpx
```

Esto simplifica el desarrollo, pero puede provocar que en el futuro se instalen versiones más nuevas con cambios incompatibles.

Para un entorno más controlado, podrían fijarse versiones concretas:

```
fastapi==0.128.0
uvicorn==0.38.0
sqlalchemy==2.0.45
```

En un MVP académico, no es necesariamente un problema, pero conviene saberlo.

♦ Error instalando `psycopg`

Si se usara solo:

```
psycopg
```

podrían aparecer problemas relacionados con librerías del sistema.

Al usar:

```
psycopg[binary]
```

se reduce ese riesgo porque se instala la variante binaria preparada.

♦ Tests que fallan por falta de `httpx`

En proyectos FastAPI, algunos tests dependen de `httpx` para simular peticiones HTTP.

Si `httpx` no está instalado, los tests podrían fallar aunque el backend esté bien.

1 1 Comandos útiles relacionados

Instalar dependencias localmente desde la carpeta backend:

```
cd ~/siem-lab/backend
pip install -r requirements.txt
```

Ver dependencias instaladas dentro del contenedor:

```
docker exec -it siem-api pip list
```

Comprobar si FastAPI está instalado:

```
docker exec -it siem-api python -c "import fastapi; print(fastapi.__version__)"
```

Comprobar si SQLAlchemy está instalado:

```
docker exec -it siem-api python -c "import sqlalchemy; print(sqlalchemy.__version__)"
```

Comprobar si Psycopg está instalado:

```
docker exec -it siem-api python -c "import psycopg; print(psycopg.__version__)"
```

Ejecutar tests dentro del backend:

```
cd ~/siem-lab/backend
pytest
```

Ejecutar tests dentro del contenedor:

```
docker exec -it siem-api pytest
```

Reconstruir la imagen después de modificar dependencias:

```
docker compose --env-file docker/.env -f docker/compose.yml build api
```

Levantar de nuevo la API:

```
docker compose --env-file docker/.env -f docker/compose.yml up -d api
```

1 2 Cómo explicarlo en la presentación

El archivo `backend/requirements.txt` define las dependencias Python necesarias para ejecutar el backend del laboratorio. Entre ellas se encuentran FastAPI, que permite construir la API; Uvicorn, que ejecuta el servidor; SQLAlchemy, que permite trabajar con la base de datos mediante modelos; Psycopg, que actúa como driver de conexión con PostgreSQL; Alembic, que gestiona migraciones; y Pytest junto con HTTPX, que permiten ejecutar pruebas automatizadas sobre la API.

Este archivo se utiliza durante la construcción de la imagen Docker del backend. El `Dockerfile` copia `requirements.txt` dentro del contenedor y ejecuta `pip install -r requirements.txt`, dejando la imagen preparada para arrancar la aplicación FastAPI mediante Uvicorn.

Aunque no contiene lógica propia del SIEM, es una pieza esencial porque define la pila técnica que permite que el backend funcione.